

Aprendiendo Linux: Gestión de Ficheros

Sitio: [PLATEA \(Plataforma de Enseñanza y Aprendizaje de la Universidad de Jaén\)](#)

Curso: 24/25 Sistemas operativos (13312037-2425,74018016-2425)

Libro: Aprendiendo Linux: Gestión de Ficheros

Imprimido por: Díaz Galiano, Manuel Carlos

Día: viernes, 10 de enero de 2025, 00:06

Descripción

En este tema se pretende describir con rigor algunas de las órdenes de uso más frecuente en UNIX relacionadas con el sistema de

Tabla de contenidos

1. Introducción

- 1.1. Sistemas de ficheros subsidiarios
- 1.2. Nodos i
- 1.3. Directorios y enlaces
- 1.4. Apertura de un fichero

2. Enlace de ficheros

3. La orden ln

4. Traslado de ficheros: la orden mv

5. Copia de ficheros: la orden cp

- 5.1. Copiado no recursivo
- 5.2. Copiado recursivo

6. Eliminación de ficheros y directorios: las órdenes rm y rmdir

- 6.1. Eliminación de ficheros: la orden rm
- 6.2. Eliminación de directorios: la orden rmdir

7. Creación de directorios: la orden mkdir

8. Localización de ficheros: la orden find

9. Ejemplos

10. Ejercicios propuestos

1. Introducción

Autor: Lina García Cabrera & Francisco Martínez del Río

Copyright: 

Este material está sometido a las condiciones de una [Licencia Creative Commons Attribution-NonCommercial-No Derivative Works 3.0](#).

En este módulo se pretende describir con rigor algunas de las **órdenes de uso** más frecuente en UNIX relacionadas con **el sistema de ficheros**. Éstas permiten realizar tareas como **copiar, mover, crear, borrar ficheros y directorios** dentro de la estructura jerárquica o **buscar la ubicación de un fichero** dentro de la jerarquía de ficheros.

En esta sección se describen algunos conceptos de UNIX, como **los enlaces**, claves para entender este módulo.

1.1. Sistemas de ficheros subsidiarios

UNIX permite que **ciertas partes de la jerarquía de ficheros**, llamadas **sistemas de ficheros**, residan en **dispositivos** de almacenamiento o **particiones de disco aparte**. Hay dos razones para hacerlo:

- En primer lugar, el tamaño del disco u otro dispositivo que contenga los directorios y ficheros puede ser insuficiente para almacenar toda la jerarquía, siendo necesario distribuir la jerarquía en varios dispositivos.
- En segundo lugar, los dispositivos de almacenamiento quizá no estén conectados al ordenador de forma permanente. Un ejemplo de sistema de ficheros es una [memoria flash](#) (y los computadores antiguos, el [disco flexible](#)) que contiene su propia estructura de ficheros.

UNIX gestiona los sistemas de ficheros relacionándolos con **puntos de montaje**. Un punto de montaje **es un directorio en un sistema de ficheros que se corresponde con el directorio raíz de otro sistema de ficheros**.

- El **sistema de ficheros primario** es el que surge del directorio raíz verdadero y se denomina `/`.
- Los **sistemas de ficheros secundarios** se enlazan con el sistema primario a través de la orden **mount**, a la que se le da la trayectoria del [punto de montaje](#) y la ubicación del sistema de ficheros secundario. Su efecto es hacer que la raíz del sistema de ficheros secundario corresponda al punto de montaje. Una vez que se ha montado un sistema de ficheros, se puede hacer referencia a cualquier fichero o directorio que contenga utilizando una trayectoria que pase por el punto de montaje. El desmontado (mediante la orden **umount**) de un sistema de ficheros invalida todas las trayectorias que pasan por el punto de montaje.

Usted, como usuario, debe ser consciente de los límites del sistema de ficheros por dos razones:

- En primer lugar, existen **restricciones para crear enlaces fuera de un sistema de ficheros** (lo veremos a continuación).
- En segundo lugar, **cuando se desmonta un sistema de ficheros se deja de tener acceso a todos los ficheros** que contiene hasta que se vuelve a montar.

NOTA: En Linux, la orden **mount** muestra todas los montajes activos en el sistema, no solo las unidades físicas o de almacenamiento, sino también sistemas de archivos virtuales para exponer el estado del sistema, puntos de montaje temporales que crean sistemas de archivos en RAM, sistema para instalación de paquetes snap, montajes para depuración y seguridad.

1.2. Nodos i

En UNIX todo **fichero** tiene asociado una pequeña zona de información en el disco (una estructura de datos) denominada **nodo i**. Cada archivo **se identifica por un número de inodo**. Este **número es único** dentro de todo el sistema de archivos.

Cuando se crea un nuevo archivo a este se le asigna un nombre, y además, un InodeNumber, para que el sistema operativo pueda identificar tu archivo.

Un nodo *i* contiene toda la información importante para gestionar un fichero, entre otros:

- la **identidad del propietario** del fichero y del grupo (su *uid* y *gid*)
- los **permisos** del fichero
- el **tamaño**
- la **fecha de creación y de última modificación**
- las **zonas del disco** donde se ubica el fichero
- la cantidad o **números de enlaces duros** al fichero

Mostrar el inodo de un archivo

Para saber el número de inodo de un archivo, puedes ejecutar la orden `ls` con la opción `-i` o también `--inode`. En el caso de un directorio se usará `ls -di`

```
ls -i
```

```
ls -di
```

La ejecución de este comando listará los archivos contenidos dentro de el directorio, junto con el Inode Number de cada uno de ellos.

```
1 [davidochobits@servcentos]$ ls -li
2 total 8148
3 5412 -rw-rw-r-- 1 david david 0 Apr 8 10:34 file1.txt
4 5413 -rw-rw-r-- 1 david david 0 Apr 8 10:34 file2.txt
5 5414 -rw-rw-r-- 1 david david 0 Apr 8 10:34 file3.txt
6 18 drwxr-xr-x 19 root root 4096 Mar 20 11:28 openssl-1.1.1
7 16 -rw-rw-r-- 1 david david 8337920 Sep 11 2018 openssl-1.1.1.tar.gz
```

Comprobar el uso de inodos

Cada sistema de ficheros tiene su conjunto de nodos *i* para guardar la información asociada a los ficheros del sistema. Cada sistema de ficheros tiene un número limitado de identificadores disponibles (Inode numbers), que impone el número máximo de archivos que se pueden crear que no depende del espacio en disco duro. Los nodos *i* se numeran del 0 hacia delante.

Para conocer la cantidad de números disponibles, cuántos has usado y el porcentaje que se ha ocupado:

```
df -i
```

El comando `df` muestra información acerca del espacio en disco usado por el sistema de archivos, o en palabras, el espacio en el disco duro. La opción `-i` le indica que en lugar de mostrar la información del espacio usado, muestre información relacionada con los inodos.

Esto quiere decir que nuestra computadora se puede quedar sin espacio de dos formas:

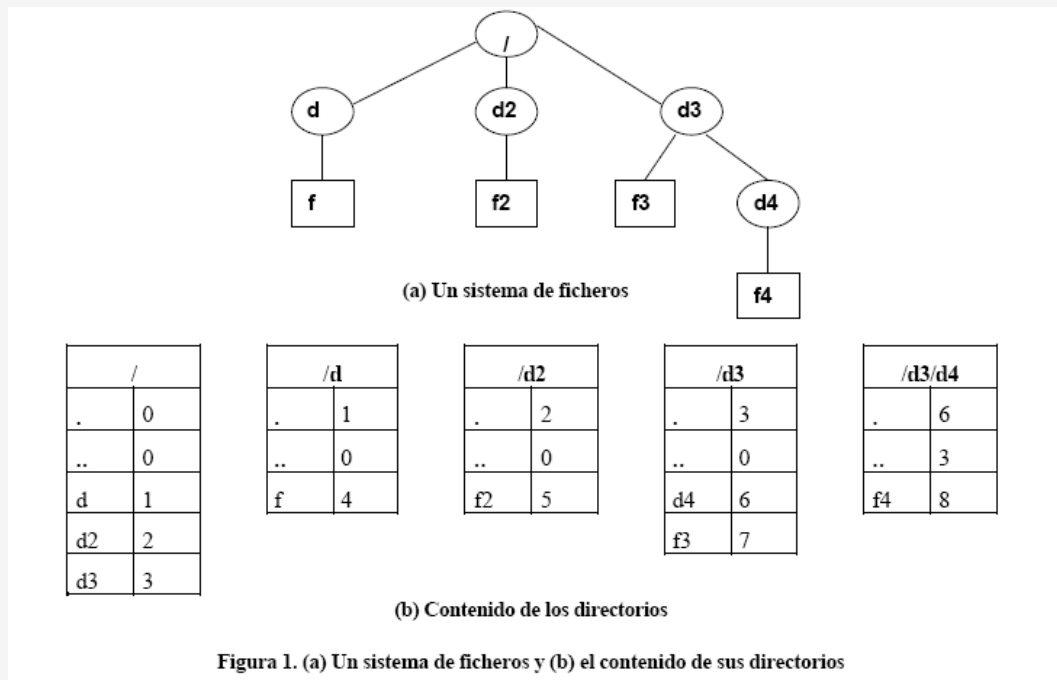
- Ocupamos toda la capacidad del disco duro
- Nos acabamos los inodos disponibles para nuestro sistema operativo

```
1 [davidochobits@servcentos /]$ df -i
2 Filesystem          Inodes IUsed  IFree IUse% Mounted on
3 /dev/mapper/vg_root-lv_root 327680 85831 241849 27% /
4 devtmpfs            482103 401 481702 1% /dev
5 tmpfs               485100 1 485099 1% /dev/shm
6 tmpfs               485100 670 484430 1% /run
7 tmpfs               485100 16 485084 1% /sys/fs/cgroup
8 /dev/sda1            65536 342 65194 1% /boot
9 /dev/mapper/vg_root-lv_var 327680 3739 323941 2% /var
10 /dev/mapper/vg_root-lv_tmp 131072 22 131050 1% /tmp
11 /dev/mapper/vg_root-lv_home 327680 5623 322057 2% /home
12 /dev/mapper/vg_root-lv_opt 327680 9988 317692 4% /opt
```

1.3. Directorios y enlaces

En UNIX un **directorio** es un **tipo especial de fichero** que contiene una serie de **entradas** llamadas **enlaces a ficheros**. Un **enlace es un par formado por un nombre**, que nombra al enlace en el directorio, y **un número de nodo i**, que indica con qué fichero está asociado el enlace (nos dice la estructura de datos que contiene información sobre el fichero).

Por ejemplo, la figura 1.b) muestra el contenido de los directorios del sistema de ficheros representado en la figura 1.a).



1.4. Apertura de un fichero

- Los procesos solicitan el uso de los ficheros mediante una llamada al sistema. A esta solicitud se le llama **apertura del fichero**. La llamada precisa como parámetro una **trayectoria o ruta del fichero** que se quiere abrir.
- Como respuesta a la solicitud el sistema operativo **localiza el nodo i del fichero en el disco** y, si el proceso tiene permisos para abrir el fichero, **carga su nodo i en memoria principal**.
- A partir de entonces, cuando el proceso solicite leer y/o escribir del fichero el sistema operativo obtiene la información necesaria para realizar la lectura o escritura del nodo i almacenado en memoria principal, en lugar de hacerlo de disco, ya que el tiempo de acceso a memoria principal es inferior al tiempo de acceso a disco.

Veamos ahora **cómo localiza el sistema operativo el nodo i de un fichero a partir de la trayectoria proporcionada por un proceso**.

Por ejemplo, supongamos que se emplea la trayectoria **/d2/f2** para el sistema de ficheros de la figura.

- Como la trayectoria comienza por **/** el sistema operativo inicia su búsqueda en el **directorio raíz**, el cual tiene asociado un nodo i hijo: el 0.
- Se consulta el nodo i número 0 para saber a partir de qué dirección de disco se almacena **/**, tras obtener la dirección se lee el contenido de **/** (obsérvese en la figura 1.b), y se busca la entrada de nombre **d2**.
- Se accede ahora al nodo i de **d2** (el 2) para localizar a partir de qué dirección de disco se almacena su contenido.
- Una vez descubierto se lee **d2** y se busca la entrada de nombre **f2**. Ésta está asociada al nodo i 5.
- Se consultan en el nodo i 5 los permisos del fichero, y si el usuario que ejecuta el proceso tiene permitido el acceso al fichero se guarda el nodo i en memoria principal.

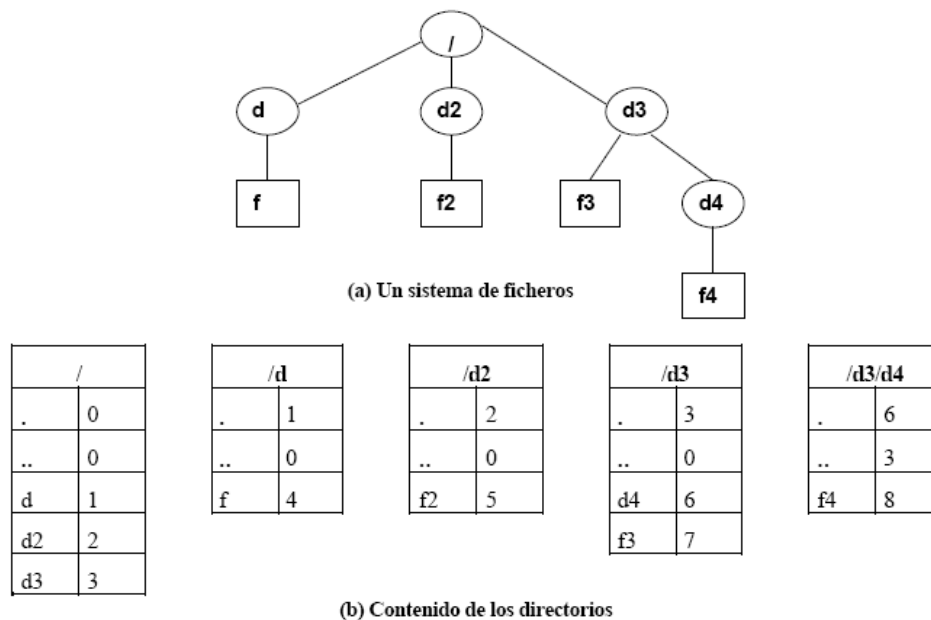


Figura 1. (a) Un sistema de ficheros y (b) el contenido de sus directorios

Con las trayectorias relativas se trabaja de forma análoga, sólo que la búsqueda comienza en el **directorio de trabajo** del proceso, en lugar de comenzar en el directorio raíz. El nodo *i* asociado al directorio de trabajo de un proceso se almacena en su descriptor o bloque de control de proceso BCP.

En la figura 1 puede observar que todo directorio posee al menos dos entradas: **.** y **..**, que hacen referencia al **propio directorio** y al **padre de éste respectivamente**. Estas entradas se añaden automáticamente cuando se crea un directorio. Su utilidad reside en permitir **construir trayectorias relativas al directorio de trabajo y al directorio padre** de éste.

Por ejemplo, si el directorio de trabajo es **/d3/d4** la trayectoria **../f3** hace referencia a **/d3/f3**. Piense que si el directorio de trabajo está situado a cierta profundidad en el sistema de ficheros y queremos utilizar una entrada de un "directorio hermano" al directorio de trabajo el uso de una trayectoria que incluya el **..** será mucho más corta que la trayectoria absoluta.

2. Enlace de ficheros

UNIX permite que exista más de un enlace a un fichero o directorio, lo que posibilita el acceso a un mismo fichero a través de distintas trayectorias. Existen dos tipos de enlaces: los enlaces tradicionales o **enlaces duros** (hard o físicos) y los **enlaces simbólicos** (soft o suaves).

La figura 2.a) representa el mismo sistema de ficheros que el de la figura 1.a), salvo que se han añadido dos enlaces más: **ed** y **es**.

- **ed** es un **enlace duro** al fichero asociado al enlace **f2** del directorio **/d2**, es decir, al fichero cuyos datos se almacenan en el nodo *i* 5. Como puede observar tanto la entrada **ed** del directorio **/d** como la **f2** del directorio **/d2** hacen referencia al mismo nodo *i*: el cinco.
- Si se tiene en cuenta lo que hace UNIX en el proceso de apertura de un fichero (comentado en la sección anterior) se deduce que la apertura de las trayectorias **/d/ed** y **/d2/f2** producen el mismo resultado: solicitar el fichero asociado al nodo *i* 5. En terminología UNIX este fichero posee dos enlaces duros.

Todos los enlaces del sistema de ficheros de la figura 2.a), salvo **es**, son duros. Observe que, por ejemplo, el directorio **/d3** posee 3 enlaces duros: el enlace **d3** de **/**, el enlace **.** de **/d3** y el enlace **..** de **/d3/d4**.

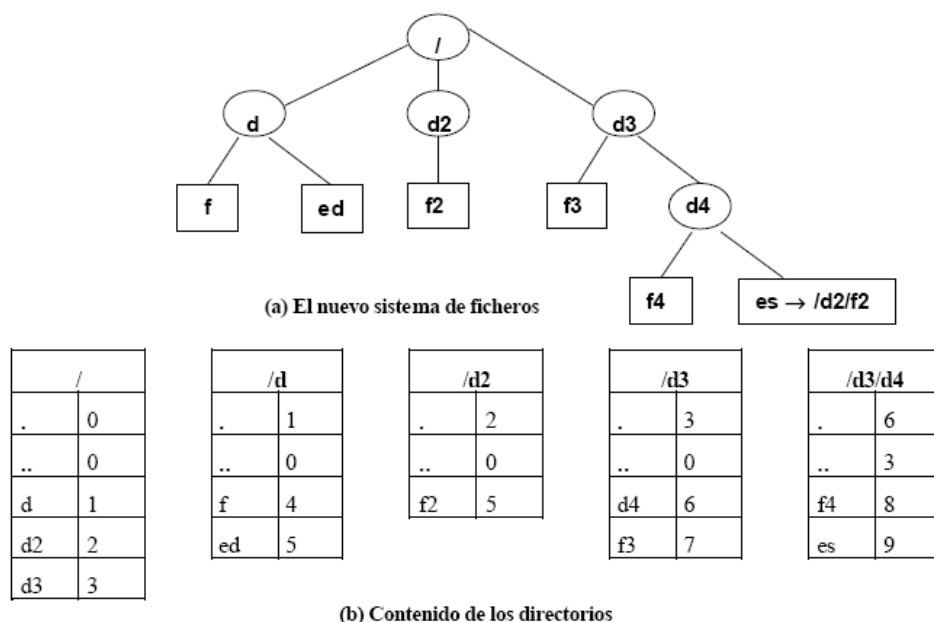


Figura 2. (a) El nuevo sistema de ficheros y (b) el contenido actualizado de sus directorios

El nodo *i* de un fichero almacena el número de enlaces duros que posee el fichero.

Los **enlaces simbólicos**, como los duros, sirven para aportar trayectorias adicionales a un fichero, pero se implantan de manera distinta, por lo que exhiben propiedades diferentes. Un enlace simbólico hace referencia a un pequeño fichero cuyo contenido es una trayectoria del sistema de ficheros.

Por ejemplo, en la figura 2.a) se hace referencia a un fichero que se almacena en el nodo *i* 9, de contenido **/d2/f2**. Cuando se solicita la apertura de una trayectoria como **/d3/d4/es** UNIX procede de forma normal, sin embargo, cuando localiza su nodo *i* asociado (el 9) se comprueba (leyendo un bit del nodo *i*) si el fichero se utiliza como enlace simbólico. Si es así UNIX, lee su contenido para obtener la trayectoria enlazada, en el ejemplo **/d2/f2**, y localizar su nodo *i*, es decir, el 5. El resultado final es acceder al nodo *i* 5, y por tanto, abrir el mismo fichero que se hubiera obtenido empleando la trayectoria **/d2/f2**. Sin embargo, observe que aunque el proceso de apertura de **/d2/f2** y **/d3/d4/es** produce el mismo resultado, la segunda trayectoria implica una apertura más lenta, pues implica más accesos a disco por parte del sistema operativo.

Otra diferencia entre enlaces duros y simbólicos viene dada por la posibilidad de realizar enlaces a sistemas subsidiarios. No es posible crear un enlace duro a un fichero de un sistema subsidiario. Esto se debe a que **cada sistema de ficheros numera su conjunto de nodos *i* del cero en adelante**. Si creamos un enlace duro al nodo *i* 7 nos referimos al nodo *i* 7 de nuestro sistema de ficheros. Sin embargo, podemos crear un enlace simbólico a un fichero de un sistema subsidiario usando una trayectoria de enlace que pase por un punto de montaje.

3. La orden **ln**

La orden **ln** (de *link*) crea un enlace con uno o más ficheros existentes. Tenemos dos formatos de uso:

\$ ln [-s] [opciones] fich1_destino fich2_enlace

\$ ln [-s] [opciones] destino1 destino2 ... dir;

En la primera forma se crea un enlace entre *fich2* y el fichero enlazado a *fich1*. Si *fich2* ya era un enlace a un fichero se destruye el enlace antiguo al crear el nuevo.

En la segunda forma se crea un enlace en *dir* con un fichero de nombre *fichero*. En este caso, **ln** crea un enlace para cada fichero en el directorio *dir*, asignando al enlace el mismo nombre que tiene el fichero. Como sucede con la primera forma de **ln**, se destruye todo enlace existente que tenga el mismo nombre. **Los enlaces duros normalmente solo pueden enlazar archivos, no directorios ni carpetas.**

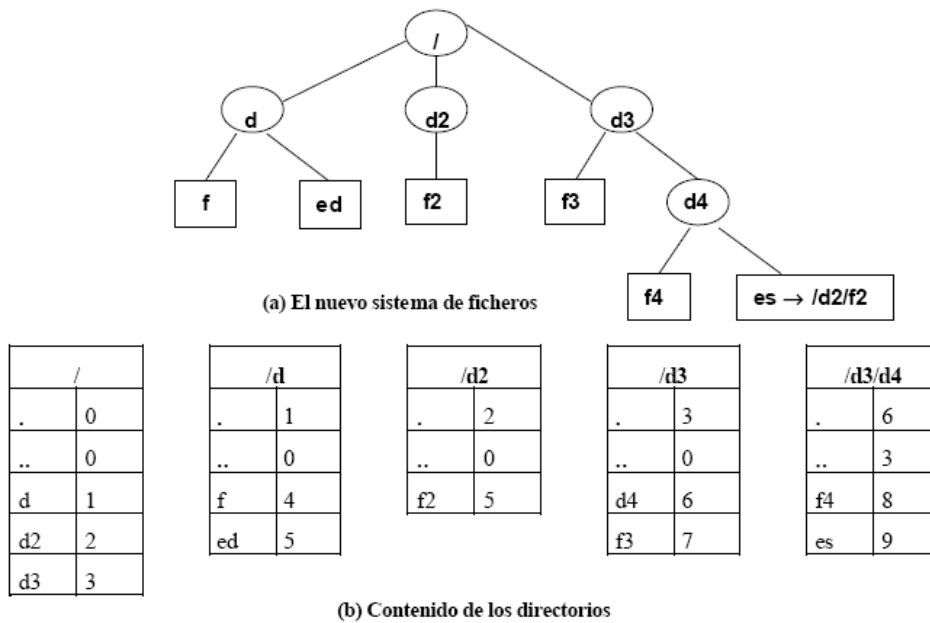


Figura 2. (a) El nuevo sistema de ficheros y (b) el contenido actualizado de sus directorios

Cuando **ln** crea un enlace, el nombre del enlace destino tiene el mismo conjunto de permisos y el mismo propietario y grupo que el enlace del fichero fuente.

No se permite crear un enlace entre un directorio de un sistema de ficheros y un fichero de otro sistema, a menos que el enlace sea simbólico. Por omisión **ln** crea enlaces duros y solicita confirmación cuando reemplaza un enlace con un fichero que no tiene permiso de escritura. Se puede cambiar este comportamiento con las siguientes opciones de la línea de órdenes:

- **-s** Crea enlaces simbólicos en lugar de enlaces duros.
- **-f** No solicita confirmación, incluso si se reemplaza un enlace sin permiso de escritura.

Crear un enlace duro

```
# ls -l /home/bobbin/sync.sh
```

```
-rw-r----- 1 root root 5 Apr 7 06:09 /home/bobbin/sync.sh
```

```
# ln /home/bobbin/sync.sh synchro
```

Ahora comparemos los dos archivos

```
# ls -il /home/bobbin/sync.sh synchro
517333 -rw-r----- 2 root root 5 Apr 7 06:09 /home/bobbin/sync.sh
517333 -rw-r----- 2 root root 5 Apr 7 06:09 synchro
```

Lo interesante de los enlaces duros es que no hay diferencia entre el archivo original y el enlace: son solo dos nombres conectados al mismo inodo.

Crear un enlace simbólico

```
# ln -s /home/bobbin/sync.sh filesync
```

Aquí 'filesync' es un enlace simbólico a 'sync.sh'. Piense en ello como un atajo. Editar 'sincronización de archivos' es como editar directamente el archivo original, pero es realmente lo que sucede. Si eliminamos o movemos el archivo original, el enlace se romperá y nuestro archivo 'filesync' ya no estará disponible.

Si se ejecuta **ls -l** se muestra que el archivo resultante es un enlace simbólico. Esto se indica con la letra **l** en la primera posición de la **ls -l** y también por la flecha al final de la lista, que indica el archivo al que se refiere el nombre.

```
# ls -l filesync
lrwxrwxrwx 1 root root 20 Apr 7 06:08 filesync -> /home/bobbin/sync.sh
```

El contenido de un enlace simbólico es solo el nombre del archivo de destino. Puede ver que los permisos en el enlace simbólico están completamente abiertos.

Al comparar el enlace simbólico y el archivo original, notará una clara diferencia entre ellos.

```
# ls -il /home/bobbin/sync.sh filesync
258674 lrwxrwxrwx 1 root root 20 Apr 7 06:08 filesync -> /home/bobbin/sync.sh
517333 -rw-r----- 1 root root 5 Apr 7 06:09 /home/bobbin/sync.sh
```

Tienen un número de inodo diferente. El archivo enlace contiene el camino del fichero enlazado, su tamaño es la cantidad de bytes del nombre del fichero al que se hace referencia.

4. Traslado de ficheros: la orden mv

La orden **mv** pasa uno o más ficheros de un directorio a otro. Si los dos directorios están en el mismo sistema de ficheros, lo único que hace **mv** es cambiar sus enlaces, sin transferir datos. Si están en sistemas de ficheros distintos, **mv** copia los datos de los ficheros del primer sistema en el segundo, crea los enlaces necesarios en el segundo sistema de ficheros, y luego elimina los enlaces en el primer sistema. El traslado se puede considerar como un cambio de nombre de los ficheros. Cuando se traslada un fichero permanecen intactos su modo de protección y su pertenencia, a menos que el movimiento sea de un sistema de ficheros a otro; en este caso, los ficheros pertenecen al usuario que ejecutó la orden **mv**. Las formas de utilizar **mv** en la línea de órdenes son:

\$ mv [opciones] fich1 fich2; \$ mv [opciones] fichero ... dir

En la primera forma se crea un enlace entre *fich2* y el fichero enlazado a *fich1*, y se elimina el enlace original de *fich1*. Si *fich2* ya estaba enlazado a un fichero, se destruye el enlace anterior al crearse el nuevo (esto no es lo mismo que destruir el fichero, ya que se conservan sus demás enlaces).

En la segunda forma, cada fichero en fichero ... se cambia a *dir*. El traslado conserva los nombres de los ficheros que se han movido.

Por omisión, **mv** solicita confirmación cuando reemplaza un enlace con un fichero que carece de permiso de escritura. Se puede modificar este comportamiento con las siguientes opciones de la línea de órdenes:

- **-i** Solicita confirmación al reemplazar cualquier enlace, sin importar sus permisos (sólo BSD UNIX).
- **-f** No solicita confirmación al reemplazar un enlace que carece de permiso de escritura.

5. Copia de ficheros: la orden cp

La orden **cp** copia uno o más ficheros. A diferencia de **ln** y **mv**, **cp** sí copia los datos y no sólo reacomoda las entradas en el directorio. Los formatos de **cp** de la línea de órdenes son:

\$ cp [opciones] fichentrada fichsalida; \$ cp [opciones] nomentrada ... dirsalida

donde *fichentrada* es un fichero de entrada, *fichsalida* es un fichero de salida, cada *nomentrada* es un fichero o directorio de entrada y *dirsalida* es un directorio de salida. Los elementos de entrada se denominan "fuentes"; los elementos de salida se conocen como "destinos".

- El primer formato se aplica cuando el destino es un fichero. En este caso sólo puede haber una fuente, un fichero. El fichero de entrada se copia en el fichero de salida.
- El segundo formato se aplica cuando el destino es un directorio.

Las copias pueden ser recursivas o no recursivas, aunque la versión de **cp** en System V sólo permite copias no recursivas. La opción **-r** especifica la recursividad de la copia, la cual sólo afecta a la copia de directorios.

5.1. Copiado no recursivo

\$ cp [opciones] fichentrada fichsalida; \$ cp [opciones] nomentrada ... dirsalida

Para una copia no recursiva es necesario que exista el directorio *dirsaldida* y que cada nomenclatura nombre un fichero, y no un directorio. Cada uno de los ficheros de entrada se copia en el directorio de salida y se le asigna el mismo nombre base que tenía originalmente. Por ejemplo, suponga que */usr/homer* y *burt* son directorios y que */usr/homer* contiene los ficheros *lisa* y *marge*. Entonces, la orden

\$ cp /usr/homer/* burt copia */usr/homer/lisa* en un fichero de nombre *burt/lisa* y */usr/homer/marge* en un fichero llamado *burt/marge*.

Si el fichero de destino no existe, se crea un fichero con ese nombre. Si el fichero de destino ya existe, se elimina el enlace especificado por el fichero de destino y se crea un enlace nuevo. Enseguida se copian los datos de cada fichero fuente en su fichero de destino correspondiente. Después de la copia, los ficheros fuente y destino son independientes, o sea, las modificaciones de un fichero no afectan al otro.

Un fichero creado por **cp** adquiere el dueño, el modo y los permisos del fichero de destino si dicho fichero existía antes de la copia; de lo contrario se asignan los del fichero fuente. Si una operación de copia va a destruir un enlace existente con un fichero con permiso de escritura desactivado, **cp** presentará el modo de protección del fichero y solicitará confirmación de la copia. Si se teclea cualquier cosa que comience con 'y' se permitirá la copia; cualquier otra cosa la cancelará.

Múltiples formas de tratar la sobreescritura mientras se copian los archivos

Probablemente no siempre querrás que tus archivos de destino existentes se sobrescriban y eso es totalmente lógico.

Para evitar la sobreescritura de archivos existentes, puedes utilizar la opción **-n**. De esta manera, **cp** no sobrescribirá los archivos existentes.

cp -n archivo_origen directorio_objetivo

Pero tal vez quieras sobrescribir algunos archivos. Puedes utilizar la opción interactiva **-i** y te preguntará si quieres sobrescribir los archivos existentes.

cp -i archivo_origen directorio_objetivo

cp: overwrite 'directorio_objetivo/archivo_origen'?

Puedes introducir **y** para sobrescribir el archivo existente o **n** para no sobrescribirlo.

También hay una opción para hacer copias de seguridad automáticas. Si utiliza la opción **-b** con el comando **cp**, sobrescribirá los archivos existentes, pero antes creará una copia de seguridad de los archivos sobrescritos.

cp -b file.txt directorio_objetivo/file.txt

ls directorio_objetivo

file.txt file.txt~

La copia de seguridad del archivo termina con **~**.

También puedes utilizar la opción de actualización **-u** cuando se trata de sobrescribir. Con la opción **-u**, los archivos de origen sólo se copiarán en la nueva ubicación si el archivo de origen es más reciente que el archivo existente o si no existe en el directorio de destino.

Para resumir:

-i : Confirmar antes de sobrescribir

-n : No sobrescribir

-b : Sobrescribir con la copia de seguridad

-u : Sobrescribir si el archivo de destino es antiguo o no existe

5.2. Copiado recursivo

\$ cp [opciones] fichentrada fichsalida; \$ cp [opciones] nomenclatura ... dirsaldida

El comportamiento de la copia recursiva depende de la existencia del directorio de salida *dirsaldida*:

- Si *dirsaldida* no existe, la entrada debe consistir en un sólo directorio. Después de crear *dirsaldida*, **cp** copia todos los ficheros y subdirectorios del directorio de entrada en *dirsaldida* para que sea una copia del directorio de entrada.

- Si *dirsalida* ya existe, cada fichero de entrada se copia en el directorio de salida de la misma manera que se hace en la copia no recursiva. Los directorios de entrada se reproducen como subdirectorios del directorio de salida, incluyendo los ficheros y subdirectorios que contengan.

Cómo copiar un directorio en Linux

También puedes utilizar el comando **cp** para copiar un directorio en Linux incluyendo todos sus archivos y subdirectorios. En este caso hay que utilizar la opción **-r** que significa recursivo.

cp -r directorio_origen directorio_objetivo

Esto copiará todo el directorio_de_origen en el directorio_de_objetivo. Ahora el directorio_fuente será un subdirectorio del directorio_objetivo.

Cómo copiar sólo el contenido de un directorio, no el propio directorio

ls directorio_objetivo

directorio_origen

En el ejemplo anterior, ha copiado todo el directorio en una nueva ubicación.

Pero si sólo quieres copiar el contenido del directorio de origen en el directorio de destino, debe añadir **/.** al final del directorio de origen. Esto indicará que sólo quieres copiar el contenido del directorio de origen.

Veámoslo con un ejemplo:

ls directorio_origen

archivo_origen_1 archivo_origen_2

Ahora copia el contenido del directorio de origen:

cp -r directorio_origen/. directorio_objetivo

Si ahora compruebas el contenido del directorio de destino, verás que sólo se ha copiado el contenido del directorio de origen.

ls directorio_objetivo

archivo_origen_1 archivo_origen_2

Cómo copiar múltiples directorios

También puedes copiar varios directorios a la vez con el comando **cp** en Linux.

Sólo tienes que utilizarlo de la misma manera que lo hiciste para un solo directorio.

cp -r directorio_origen_1 directorio_origen_2 directorio_origen_3 directorio_objetivo

Siempre es el último argumento del comando el que se toma como directorio de destino.

Si quieres copiar sólo el contenido de varios directorios a la vez, también puedes hacerlo:

cp -r directorio_origen_1/. directorio_origen_2/. directorio_origen_3/. directorio_objetivo

cp -r directorio_origen directorio_objetivo

Esto copiará todo el directorio_de_origen en el directorio_de_objetivo. Ahora el directorio_fuente será un subdirectorio del directorio_objetivo.

Cómo conservar los atributos al copiar

Cuando se copia un archivo a una nueva ubicación, sus atributos, como los permisos y las marcas de tiempo del archivo, se modifican.

Si deseas conservar los atributos del archivo original, puede copiar los archivos con la opción **-p**.

Veámoslo con un ejemplo.

ls -l /etc/services

```
-rw-r--r-- 1 root root 19183 Jul 23 2022 /etc/services
```

Si intentas copiar este archivo normalmente, sus atributos cambiarán:

```
cp /etc/services .
```

```
ls -l services
```

```
-rwxrwxrwx 1 marcocarmonaga marcocarmonaga 19183 Jul 23 20:45 service
```

Pero si uso la opción `p`, el archivo copiado conservará el modo, la propiedad y la marca de tiempo.

```
cp -p /etc/services .
```

```
ls -l services
```

```
-rw-r--r-- 1 marcocarmonaga marcocarmonaga 19183 Jul 23 2022 services
```

Como puedes ver, ha conservado el modo de acceso y la marca de tiempo del archivo fuente con la opción `-p`.

6. Eliminación de ficheros y directorios: las órdenes `rm` y `rmdir`

Las dos órdenes para eliminar ficheros son **`rm`** y **`rmdir`**. **`rm`** puede eliminar ficheros ordinarios y directorios, pero **`rmdir`** sólo puede eliminar directorios. Por otra parte, **`rmdir`** puede eliminar directorios padre que queden vacíos como resultado de eliminaciones anteriores, algo que no puede hacer **`rm`**.

La "eliminación" de un fichero, sea un fichero ordinario o un directorio, significa eliminar enlace de un directorio y no eliminar el fichero en sí. Si hay otros enlaces duros con el fichero, se conservan, lo mismo que el fichero. Un fichero sólo se elimina cuando se borra su último enlace duro. Como un directorio es un tipo especial de fichero, está sujeto a las mismas reglas.

6.1. Eliminación de ficheros: la orden `rm`

La orden **`rm`** elimina los enlaces con los ficheros, sean ordinarios o directorios. Un fichero se elimina cuando se han borrado todos sus enlaces duros, de manera que la eliminación del único enlace con un fichero borra al propio fichero. La sintaxis de **`rm`** es:

```
$ rm [opciones] fichero ...
```

donde cada fichero en *fichero ...* puede incluir enlaces a directorios si está presente la opción **`-r`**. Por omisión, **`rm`** pide una confirmación si se intenta eliminar un enlace con un fichero que no tiene permiso de escritura. La pregunta de confirmación indica los permisos del fichero como un número octal de tres dígitos. Se puede cambiar este comportamiento con las opciones **`-i`** y **`-f`** que se describen más adelante.

`rm` devuelve un código de salida cero si tiene éxito en la eliminación de todos los enlaces especificados, y un código distinto de cero en caso contrario.

Opciones de la línea de órdenes:

- **`-i`** Solicita interactivamente la confirmación de cada eliminación. Una respuesta que comience con 'y' o con 'Y' indica que debe eliminarse el fichero; cualquier otra respuesta indica lo contrario. **`rm`** solicita esta confirmación incluso si la entrada estándar no es un terminal.
- **`-r`** Elimina recursivamente los enlaces con directorios, es decir, elimina los enlaces con los directorios y con todos los ficheros y subdirectorios que contengan, directa o indirectamente. De hecho, esta opción elimina toda una rama del árbol de directorios. **`rm`** pide que se confirme la eliminación de enlaces sin permiso de escritura, pero esta solicitud se omite si la entrada estándar no proviene del terminal, o si se especificó la opción **`-f`**. Si usted rechaza la eliminación de un enlace, **`rm`** no eliminará los directorios que contengan ese enlace de forma directa o indirecta.
- **`-f`** No solicita la confirmación de las eliminaciones, incluso en el caso de enlaces que carecen de permiso de escritura. La opción **`-f`** tiene precedencia si se especifica junto con la opción **`-i`**.
- **Nota:** No se puede eliminar enlaces de un directorio para el cual no se tenga permiso de escritura, sin importar las opciones empleadas.

6.2. Eliminación de directorios: la orden rmdir

La orden **rmdir** elimina los enlaces con directorios. Su sintaxis es:

```
$ rmdir [opciones] dir ...
```

donde *dir ...* es una lista de directorios.

Por ejemplo, **\$ rmdir denver** hace que **rmdir** elimine el enlace al subdirectorio *denver* del directorio actual. A diferencia de **rm -r**, **rmdir** no eliminará un enlace con un directorio si éste no está vacío. **rmdir** devuelve un código de salida de cero si tiene éxito en la eliminación de todos los enlaces especificados y un código distinto de cero en caso contrario.

Opciones de la línea de órdenes:

- **-p** Si la eliminación de un enlace con un directorio hace que el padre directo quede vacío, también se elimina el enlace del directorio padre. **rmdir** sigue eliminando los enlaces con directorios vacíos, dirigiéndose hacia la raíz, hasta encontrar un directorio que no esté vacío. **rmdir** envía un mensaje al error estándar anunciando cada enlace que elimina y cada intento de eliminación que fracasa.
- **-s** Suprime los mensajes generados por **rmdir**.

No todos los sistemas cuentan con estas opciones.

7. Creación de directorios: la orden mkdir

La orden **mkdir** crea uno o más directorios nuevos. Su sintaxis es:

```
$ mkdir [opciones] dir ...
```

Aquí, cada *dir* es la trayectoria de un directorio que se desea crear. Cada directorio de nueva creación tiene dos entradas: **.** para el directorio en sí y **..** para su directorio padre. Los permisos del directorio se asignan como **777** y se modifican teniendo en cuenta el valor **umask** actual, aunque se puede especificar permisos distintos con la opción **-m**. El propietario y el grupo del nuevo directorio serán los del proceso bajo cuyo auspicio se creó el directorio. La creación de un directorio, al igual que la creación de un fichero, requiere permiso de escritura en el directorio padre.

Opciones de la línea de órdenes:

- **-m n** Asigna *n* como los permisos de cada directorio creado.
- **-p** Si en una trayectoria faltan componentes, crea los directorios intermedios necesarios.

Por ejemplo, suponga que */usr/luis* no tiene subdirectorios y que el usuario *luis* emite la orden **\$ mkdir -p ~/a/x**. Entonces, **mkdir** crea un directorio *a* intermedio con el directorio *x* como entrada inicial.

No todos los sistemas reconocen estas opciones.

Con la opción **-p** y las expresiones regulares que admite el shell, es posible crear un árbol completo de directorios con tan sólo una orden. Por ejemplo, si se ejecuta

```
$ mkdir -p Musica/{Pop,Metal/Heavy,BSO,Rock/{Gotico,Progresivo},Clasica/Barroca}
```

Se crea el siguiente árbol de directorios (para ver ese árbol puedes usar la orden **tree**, no está instalada por defecto, para instalarla **sudo apt install tree**):

Musica

├── BSO

├── Clasica

| └── Barroca

├── Metal

| └── Heavy

└── Pop

```

└─ Rock
  └─ Gotico
    └─ Progresivo

```

Comprueba el número de enlaces físicos de cada uno de los directorios creados, ¿cuántos son?, ¿por qué?

Si se ejecuta

```
$ mkdir -p Contactos / {Clientes, Proveedores} / {Pedidos, Facturas}
```

```

Contactos/
└─ Clientes
  └─ Facturas
    └─ Pedidos
  └─ Proveedores
    └─ Facturas
      └─ Pedidos

```

Comprueba el número de enlaces físicos de cada uno de los directorios creados, ¿cuántos son?, ¿sabes ya cómo calcular el número de enlaces físicos de un directorio como mínimo?

8. Localización de ficheros: la orden find

El programa **find** busca en las partes especificadas del sistema de ficheros de UNIX los ficheros que cumplen cierto criterio. Este programa tiene multitud de opciones que permiten aplicar criterios complejos como: *aquellos ficheros que ocupen más de 10 bloques y su propietario sea distinto del que ejecuta el programa find*. En esta sección sólo se comenta cómo localizar ficheros que coincidan con un patrón de búsqueda, si quiere ampliar sus conocimientos sobre **find** consulte un manual o **man**.

Para buscar los ficheros que coinciden con un patrón utilizaremos la sintaxis:

```
$ find listrayec -name patrón
```

listrayec es una lista de trayectorias de ficheros y directorios (por lo general, sólo de directorios), indica los ficheros y directorios en los que se debe realizar la búsqueda, si es un directorio la búsqueda será recursiva.

patrón indica el patrón de búsqueda que deben cumplir los ficheros para que **find** mande su trayectoria absoluta a la salida estándar. En *patrón* puede especificar comodines, tal como se explicaron en el tema segundo.

Por ejemplo, la orden **\$ find /usr -name "v*.h"** envía a la salida estándar una lista de todos los ficheros que están en */usr* o sus subdirectorios, y que además comiencen con 'v' y terminen con '.h'. La lista podría verse como sigue:

```

/usr/spool/uucppublic/src/vlimit.h
/usr/include/sys/var.h
/usr/include/sys/vt.h
/usr/include/sys/vtoc.h
/usr/include/values.h
/usr/include/varargs.h

```

Siempre que se utilicen comodines en el patrón, éste se debe entrecomillar, ya que los metacaracteres que van entre una expresión entrecomillada no los interpreta el *shell*. Con esto se logra que sea **find** la que interprete los comodines.

También se puede buscar por el **último acceso**. Se utiliza la opción **-atime** seguido por un número de días o por el número y un signo + o - delante de él.

```

-atime 7 busca los archivos a los que se accedió hace 7 días.
-atime -2 busca los archivos a los que se accedió hace menos de 2 días.
-atime +5 busca los archivos a los que se accedió hace más de 5 días.

```

Otro criterio es por el tipo, se usa la opción **-type** seguida del carácter que indica el tipo de archivo, f fichero ordinario, d directorio.

```
$ find . -type f
```

Se pueden buscar también por su tamaño en bloques con `-size` seguido de un número y un signo (+ o -).

En el siguiente ejemplo se muestra la búsqueda de los archivos de tamaño igual a 10 bloques o cuyo último acceso (modificación) se haya efectuado hace más de dos días:

```
$ find . -size 10 -o -atime +2
```

9. Ejemplos

En este apartado se muestran una serie de ejemplos que aclaran las diferencias entre enlace duro y simbólico, y el uso de algunas órdenes. En ellos se empleará el programa cambia descrito en el tema anterior.

[ncambia](#)

A continuación se muestran los ejemplos:

```
$ cd Nos desplazamos a nuestro directorio base
```

```
$ echo abc >f Creamos el fichero f con el contenido abc
```

```
$ cat f Efectivamente, f contiene
```

```
abc
```

```
$ ls -li f Consultamos información sobre el fichero asociado al enlace de nombre f
```

```
2059 -rw-r--r-- 1 x1111111 alumno 4 Oct 4 10:42 f
```

La opción **i** de **ls** hace que el listado muestre el nodo **i** (en este caso 2059) de los enlaces listados. Resulta interesante observar la columna que viene tras los permisos, ésta indica el número de enlaces duros al fichero enlazado. En este caso acaba de ser creado, y sólo posee 1 (~x1111111/f).

```
$ ln f ed Creamos un enlace duro de nombre ed al fichero que enlaza f
```

```
$ ls -li ed f Consultamos información sobre los enlaces ed y f
```

```
2059 -rw-r--r-- 2 x1111111 alumno 4 Oct 4 10:42 ed
```

```
2059 -rw-r--r-- 2 x1111111 alumno 4 Oct 4 10:42 f
```

El nodo **i**, los permisos, el número de enlaces duros, el propietario y su grupo, el tamaño y la fecha de creación son la misma, pues ambos enlaces hacen referencia al mismo fichero. Note que el contador de enlaces duros es ahora dos.

```
$ ln -s ~/f es Creamos un enlace simbólico de nombre es al fichero que enlaza ~/f
```

```
$ ls -li es f Consultamos información sobre los enlaces es y f
```

```
2065 lrwxrwxrwx 1 x1111111 alumno 16 Oct 4 10:50 es -> /home/x1111111/f
```

```
2059 -rw-r--r-- 2 x1111111 alumno 4 Oct 4 10:42 f
```

Analizando la información proporcionada por **ls** podemos deducir que **es** está enlazado con otro fichero, cuyo nodo **i** es el **2065**. Además, este fichero se utiliza para almacenar una trayectoria (el primer carácter **l** en el listado de permisos nos dice que es un enlace simbólico) y tiene un único enlace duro (~x1111111/es). Su fecha de creación es posterior a la del fichero enlazado por **f**. Observando la última columna podemos saber su contenido: `/home/x1111111/f` (su tamaño es **16**, el número de caracteres de la trayectoria que almacena).

```
$ cambia f a A Cambiamos el fichero a través de la entrada f
```

```
$ cambia ed b B Cambiamos el fichero a través de la entrada ed
```

```
$ cambia es c C Cambiamos el fichero a través de la entrada es
```

```
$ cat f ed es Observamos el contenido a través de las tres entradas
```

```
ABC
```

```
ABC
```

ABC

Como puede deducirse de esta secuencia de órdenes a través de los tres enlaces se accede al mismo fichero.

\$ rm f *Borramos el primer enlace al fichero*

\$ ls -li ed *Ahora sólo existe un enlace duro al fichero*

```
2059 -rw-r--r-- 1 x1111111 alumno 4 Oct 4 10:42 ed
```

\$ cat ed es *Intentamos acceder a través de ed y es*

ABC

```
cat: es: no existe el fichero o el directorio
```

Sólo se puede a través de *ed*, pues es intenta acceder a través de la trayectoria *~x1111111/f*, y ésta ya no existe.

\$ echo El país >f *Enlazamos f a un fichero distinto*

\$ cat f ed es

```
El país
```

ABC

```
El país
```

\$ cp f copia *Copiamos el fichero enlazado a f a otro fichero enlazado por copia*

\$ ls -li copia f *Consultamos información sobre los enlaces copia y f*

```
2066 -rw-r--r-- 1 x1111111 alumno 8 Oct 4 10:50 copia
```

```
2059 -rw-r--r-- 1 x1111111 alumno 8 Oct 4 10:42 f
```

Observe que los ficheros enlazados a *copia* y *f* tienen distinto nodo *i*, y distinta fecha de creación.

\$ cambia f l L

\$ cambia copia p P

\$ cat copia f

```
El País
```

```
EL país
```

De la última secuencia de órdenes se deduce que cuando se copia un fichero se crea un nuevo fichero, al que se le asocia un nodo *i* nuevo. Se crea una copia de los datos del fichero fuente en el destino, y las modificaciones en uno de los ficheros no afectan al otro.

10. Ejercicios propuestos

1) Por cada uno de los enunciados siguientes escriba una línea de entrada para el intérprete de órdenes que satisfaga la demanda del enunciado. (Nota: los enunciados deben satisfacerse mediante una única línea de entrada al *shell*. En general, para satisfacer el enunciado *i* es preciso que antes haya satisfecho los enunciados que le preceden).

1. Cambie el directorio de trabajo a su directorio base.
2. Muestre en la pantalla el contenido del fichero */etc/passwd* empleando una trayectoria relativa para hacer referencia al fichero.
3. Cree un directorio de nombre *temp* que cuelgue directamente de su directorio base.
4. Muestre en la pantalla el contenido del directorio creado empleando una trayectoria absoluta para hacer referencia al directorio.

5. Crea en tu directorio base el directorio *dir*.
6. Crea con solo una orden la estructura de directorios de la imagen que aparece en 1.3. Directorios y enlaces. También con solo una orden todos los ficheros que aparecen en esa estructura.
7. Muestre en la pantalla el contenido del fichero *f*. Éste cuelga del directorio *d*, que a su vez cuelga del directorio *dir* que acaba de crear en la pregunta previa.
8. Muestre en la pantalla las trayectorias de los ficheros de nombre *que comienzan por f* que existen en el árbol de directorios que comienza en su directorio base.
9. Muestre en la pantalla el número de ficheros de nombre que comienzan por *f* que existen en el árbol de directorios que comienza en su directorio base. Sugerencia: interconecte la salida de la orden que utilizó en 8) con **wc**.
10. Cree un enlace simbólico en su directorio base con al directorio *dir*.
11. Borre el directorio *dir* de su directorio base (con todo su contenido).
12. Cree el directorio *~/d2/d3/d4* con permisos *rwX-----*.
13. Copie el fichero */etc/passwd* a su directorio base.
14. Desplace (mueva) el fichero de su directorio base copiado en 13) al directorio creado en 12).
15. Muestre en la pantalla el contenido del directorio *dir* utilizando el enlace que creó en su directorio en 10).
16. Cree un enlace duro en su directorio base con el fichero que desplazó en 14).
17. Muestre el contenido del fichero asociado al enlace duro que creó en 16).
18. Ejecute el programa **creacap** (lo puedes descargar desde “preguntas frecuentes” de la carpeta “Materiales de prácticas”).
19. Desplace (o mueva) todos los ficheros (realmente son enlaces) de su directorio base cuyo nombre comience por *cap* al directorio *temp* (que creó en 3).
20. Muestre en la pantalla el contenido de los ficheros del directorio *temp* cuyo nombre tiene exactamente cuatro caracteres.
21. Muestre en la pantalla el contenido de los ficheros del directorio *temp* cuyo nombre empiece por *cap* seguido de un número entre el 1 y el 7, y que terminan en la cadena *.2*.
- 2) Tras satisfacer el enunciado 6) de la pregunta anterior, ¿cuántos enlaces duros existen al directorio *dir* de su directorio base? ¿Por qué existen ese número de enlaces duros?
- 3) Compruebe empíricamente si es posible realizar enlaces duros y simbólicos a trayectorias que no existen. ¿Por qué cree que es posible o imposible?
- 4) ¿Qué ocurre con los permisos de un fichero o directorio cuando se copia?, ¿se preservan los permisos o se aplica la máscara?, ¿de qué forma?, ¿cómo se puede hacer una copia con cp preservando los permisos?
 1. Ponga una máscara que quite los permisos de ejecución a los otros.
 2. Copia un fichero que tenga los permisos de activos de ejecución para todos.
 3. Comprueba los permisos del fichero.